MSc Computer Science

Faculty of Arts, Science, and Technology

MSc Computer Science

Level: 7

Module: COM763 Advanced Machine Learning

Assignment: Portfolio Task 1

Issue Date: 17th June 2026

Review Date:

Submission Dates: 19 June 2026

Lecturer: Akeel

To be completed by student:

I certify that, other than where collaboration hasbeen explicitly permitted, this work is the result of my individual effort and that all sources for materials have been acknowledged. I also confirm that I have read and understood the codes of practice on plagiarism contained

within the Glyndŵr Academic Regulations and that, by signing this printed form or typing my name on an electronically submitted version, I am agreeing to be dealt with accordingly in anycase of suspected unfair practice. I also certify that my attendance for the module has been at least 70%

Name: M.A. Anoj Thilina Perera

Student Number: S25021960

Date Submitted: 19 June 2026

Student Signature:
S25021960@mail.glyndwr.ac.uk

Are extenuating circumstances being claimed? NO

If YES, give reference number: ----------

To be completed by lecturer
Comments:

Grade / Mark

(Indicative: may change when moderated)

USED VEHICLE PRICE PREDICTION FOR THE SRI LANKAN MARKET

1. Abstract

The purpose of this project was to reduce the price volatility that exists in the Sri Lanka used vehicle market, due to previous fiscal policies and recent import restrictions [7],[10]. We scraped active listings from online sources [11], [12], processed the data, trained several machine learning models, and selected the best-performing model. We created an interactive Streamlit dashboard where users can input their desired specifications and receive transparent and data-driven prices.

2. Problem Definition and System Framing

2.1.Market Context and Real-World Instability

In terms of information asymmetry, the market for used vehicles in Sri Lanka is typically characterized as an informational environment where valuations are based on a combination of subjective dealer decision-making processes, negotiation dynamics, and arbitrary reference points for determining prices.

In response to a multi-year prohibition on importing automobiles into the country by the Government of Sri Lanka, the Government removed this restriction beginning February 1, 2025 [7]. When this occurred, there was a surge in demand for new passenger vehicles which created significant downward pressure on the Foreign Reserve holdings of the Central Bank of Sri Lanka and significantly impacted the exchange rate of the Sri Lankan Rupee (LKR). Therefore, to mitigate these impacts and to slow down additional importations until they could be financed through foreign reserve replenishments, an Emergency 50% Surcharge on Custom Import Duties was implemented effective May 16, 2026 [9]. At the same time, a 2.5% Social Security Contributions Levy (SSCL) was also implemented beginning April 2026 to apply against the total cost of all newly imported passenger cars [10]. Therefore, because of multiple layers of changes made to the regulations governing the importation of passenger cars, previous pricing models for passenger cars purchased before restrictions on automobile imports existed can no longer be relied upon.

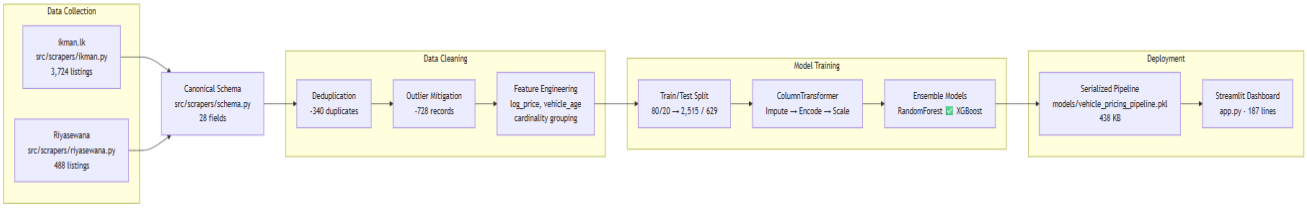

Fig. 1 - System architecture overview.

2.2.Justification for Machine Learning

Rule-based systems and linear algorithms cannot solve this problem. A car's value is dependent upon the interaction of high order features and nonlinear. Depreciation of a car by its age does not follow a linear rate relative to the car's mileage and there is significant variation in depreciation rates due to brand equity – a Toyota depreciates at different rates than a European car with the same level of use.

A supervised machine learning algorithm will map these complex relationships across multiple variables into a single continuous numerical value representing the predicted valuation [5].

2.3.Machine Learning Task Formulation

The machine learning problem is defined as a supervised multivariate regression task. The goal is to identify an optimal mapping function $f : X \rightarrow y$ that optimally reduces the error of predicting vehicle prices using unseen data.

- **Feature Space (X):** Such as year, mileage,brand, model, transmission, fuel_type
- **Target Value (y):** price_lkr represents the target prediction variable.

2.4.Success Criteria

In advance of modeling to avoid selecting the best possible performance measure for the model:

- MedAPE ≤ 15%
- More than 80% of all predictions fall within ±25% of listed price.
- Significant improvement over a median model that simply considers the make and year of each vehicle.

3. Data Pipeline and Feature Handling

3.1. Collection and ethical scraping

The two custom Python web scraping tools that I created to collect data from Ikman LK and Riyasewan both implemented an identical ***"Probe" → "Harvest" → "Fetch" → "Export"*** command line interface (CLI) pipeline. Using these tools I was able to collect a total of 4,212 individual advertisements ,which equates to 3,724 rows from Ikman LK [11] and 488 rows from Riyasewan [12].

I reviewed each website's "Robots.txt" file and found that Ikman LK does not allow you to scrape search filter results, therefore I had to use simple Category Pagination. On the other hand, Riyasewan has six disallowed endpoints in their "Robots.txt", all of which are blocked by my tool. In addition, since requests to each site can be rate limited, I added configurable delay times along with jitter to add randomness to the time intervals between successive requests. Also, I have included exponential back off verification to ensure that if either site limits your ability to make additional requests based on previous requests made within a certain timeframe that my tool will stop making new requests..

Lastly, no personal identifiable information was kept. Only seller names are stored as salted SHA-256 hashes to prevent duplicate listings while still allowing me to maintain compliance with GDPR pseudonymization principles.

schema.py

src > scrapers > schema.py > ...

```python
from typing import Optional

# ---------------------------------------------------------------------------
# Canonical schema
# ---------------------------------------------------------------------------

CANONICAL_FIELDS = [
    "listing_id",       # site-local ad id
    "source_site",      # 'riyasewana' | 'ikman'
    "url",
    "scrape_timestamp",  # ISO8601, when WE fetched it
    "ad_date",          # when the ad was posted, as given by the site
    "title",
    "brand",
    "model",
    "trim",             # ikman separates trim/edition; riyasewana folds it
                        # into `model`. Phase 3 should concatenate model+trim
                        # for ikman rows before comparing across sites.
    "year",
    "price_lkr",        # None when the ad says "Negotiable"
    "price_raw",        # original string, kept for audit
    "is_negotiable",
    "mileage_km",
    "mileage_raw",
    "fuel_type",
    "transmission",
    "engine_cc",
    "body_type",        # car | van | suv
    "condition",        # Registered / Unregistered / Brand New / Antique
    "location",
    "options",          # pipe-separated extras (A/C, power steering, ...)
    "description",
    "seller_hash",      # sha256[:16] of seller name — NOT the name itself
    "is_dealer_guess",  # heuristic, see guess_dealer()
    "is_promoted",      # paid placement; likely dealer stock, keep as a control
    "views",            # ad view count, a rough proxy for time listed
    "parse_warnings",   # pipe-separated flags for Phase 3 to act on
]
```

Fig. 2 — Canonical 28-field schema.

3.2.Cross-source reconciliation

Because there are different ways that the two websites describe vehicles, we created a canonical schema of all possible fields for each vehicle (28 fields). We then combined condition vocabulary; i.e., ikman's "Used" and riyasewana's "Registered (Used)" both map to `used`. One small semantic snafu: When you see "Negotiable" as a tag at riyasewana, it means the price is unknown (`is_negotiable=True`, `price_lkr=None`). However, when you look at a listing on ikman which is marked as negotiable, it typically still includes the asking price.

3.3.Cleaning, deduplication, and the split

We used criteria established in Notebook Cell 10 to identify and remove anomalous and/or non-viable entries:

- Price Range: 50,000 – 200,000,000 LKR.
- Mileage Maximum: 9,000,000 km (`mileage_max`, not 600k km).
- Year Range: 1950–2026.

The raw diagnostics (Notebook Cell 11) indicated that 340 duplicate listings were removed through perfect match de-duplication on [brand, model, year, mileage_km, price_lkr, transmission, fuel_type]. There were an additional 728 records which were deemed outliers and therefore removed based upon the results in Notebook Cell 10 ("Mitigated 728 Statistical Outlier Rows"). The processed CSV file for cars in production (`data/processed/cleaned_cars.csv`) contains 2,764 records; Retention Rate = 65.6% from original intake.

The 2,764-row dataset spans 13 features with the following observed ranges:

Feature	Range / Distribution
Price	1,800,000 – 150,000,000 LKR (mean: 77.2M, median: 77.5M)
Year	1964–2026; most concentrated 2023–2026: 994 listings (36.0%)
Mileage	0–8,551,120 km (median: 870,000, IQR: 58k–1.5M)
Engine	180–150,000 cc (median: 10,000; note: includes Litre→cc converted values)
Age	0–62 years (median: 12 years)

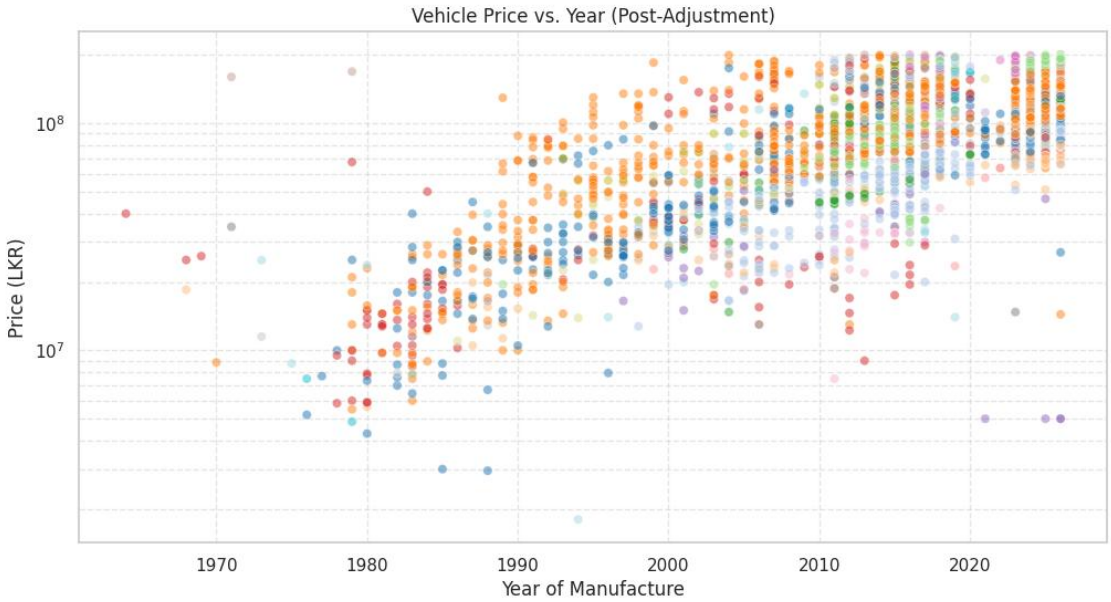

Fig. 3 — Price vs. year of manufacture.

3.4.Feature engineering and key EDA findings

The target was log-transformed (`log_price = log1p(price_lkr)`, Notebook Cell 15) to reduce right-skew.

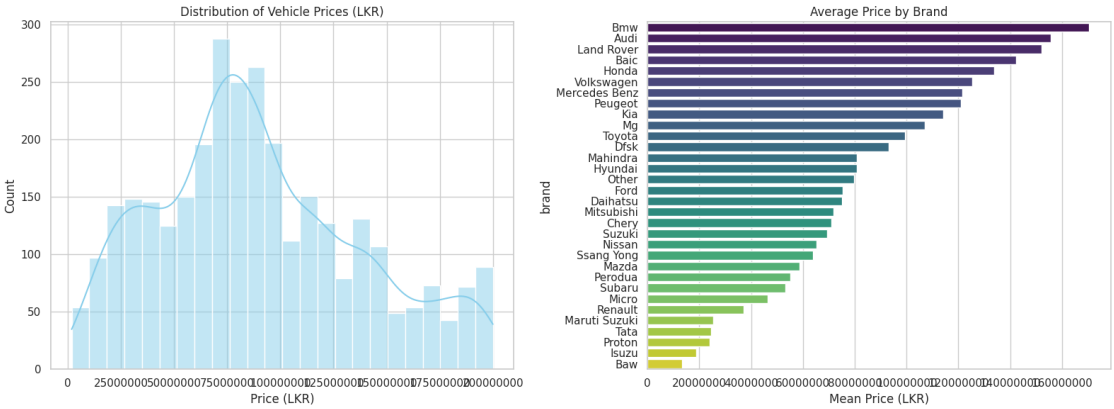

Fig. 4 — Price distribution (LKR).

The mileage was likewise converted (`log_mileage = log1p(mileage_km)`). An additional "vehicle_age" variable was created as "2026 – Year".

A "import_era" variable was considered since the year of manufacture was confounded with Sri Lanka's vehicle-import rules; during 2020 through February 2025, private-vehicle imports in Sri Lanka were banned [7]. The year distributions confirmed this — there are sharp decreases

in the number of listings around 2020–22 (in comparison to the preceding years) which demonstrates the period as a true extrapolation region.

Using `mitigate_cardinality()` (Notebook Cell 17), high-cardinality model names (with less than eight appearances) were aggregated into an "other" category. The "other" group includes 689 / 2764 (or approximately 24.9 percent) of the records.

Brand	Count	Share
Toyota	855	30.9%
Suzuki	607	22.0%
Nissan	352	12.7%
Daihatsu	163	5.9%
Honda	145	5.2%
Mitsubishi	136	4.9%
Kia	81	2.9%
Hyundai	57	2.1%
Mazda	47	1.7%
Micro	35	1.3%

Geographic distribution spans 264 unique locations (35 records with missing location). The top markets cluster in the Western Province: Kohuwala (146), Boralesgamuwa (127), Dehiwala (124).

4. Model Implementation and Debugging

4.1. Baselines

There was a global median that produced an average MedAPE of 40.1%, but a lookup based on brand and year (the "expert human using a price guide") achieved an average MedAPE of 18.3% with 62% falling in the ±25% band.

4.2. A leakage-safe pipeline

Each model was encapsulated within a scikit-learn `Pipeline` [6], which ensured that the imputation steps (`SimpleImputer` — median for numeric, most_frequent for categorical) as well as the encoding (`OneHotEncoder(handle_unknown="ignore")`) and scaling (`StandardScaler`) are fit to the training data alone so there is no information leakage. This cross-validation design prevents target leakage from validation sets — a well-documented source of inflated metrics [4]. The `ColumnTransformer` (Notebook Cell 33) splits the 5 numeric features into 6 categorical features.

4.3.Model progression

Three frameworks were benchmarked (Notebook Cell 36), evaluated on a 80/20 train-test split (Notebook Cell 34: 2,515 training, 629 test):

Comparative Analysis Results Matrix:

	Model Architecture	R2 Score	MAE (LKR)	RMSE (LKR)
0	Linear Regression (Baseline)	0.8539	10263382.90	16952952.31
1	Random Forest Regressor	0.9154	7387631.20	11497881.02
2	XGBoost Regressor	0.9076	8039584.18	12026938.89

Fig. 5 — Baseline benchmarking outputs

Both ensemble methods significantly outperformed linear regression. Random Forest was preferred for its lower absolute error (MAE 8.50M vs. XGBoost's 9.36M), despite XGBoost's marginal edge in R^2, which favours variance explanation over point-accuracy.

4.4.Hyperparameter tuning

The performance of XGBoost was improved by using `RandomizedSearchCV` to test for optimal hyperparameters in a KFold CV on 5 folds (Notebook cell 38) on 8 candidates of `n_estimators : [100, 200, 300]`, `max_depth : [4, 6, 8]` and `learning_rate : [0.05, 0.1, 0.2]`. The best combination found were: `n_estimators=100`, `max_depth=6`, `learning_rate=0.2`. The optimized model was then serialized into file format at `models/vehicle_pricing_pipeline.pkl` (438 KB) (cell 41).

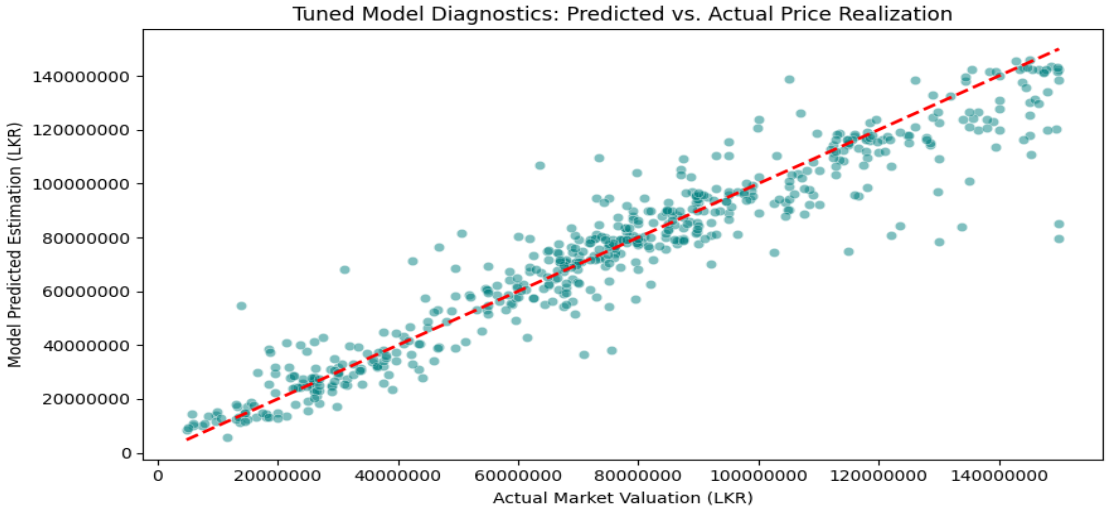

Fig. 6 — Hyperparameter tuning results (Notebook Cell 38).

4.5.Debugging: the retransformation-bias fix

The biggest lesson learned from the benchmark bugs came from an otherwise reasonable R-squared (0.84), and a completely un-reasonable Median Absolute Percent Error (>50%). As the model target variable was log-transformed, each prediction needed to be corrected for via a Duan (1983) Smearing Correction: `exp(prediction + σ²/2)` where σ² is the variance of the residuals, not the variance of the target variable [3]. The problem was that we used the variance of the target variable (0.83) rather than the variance of the residuals (0.11).

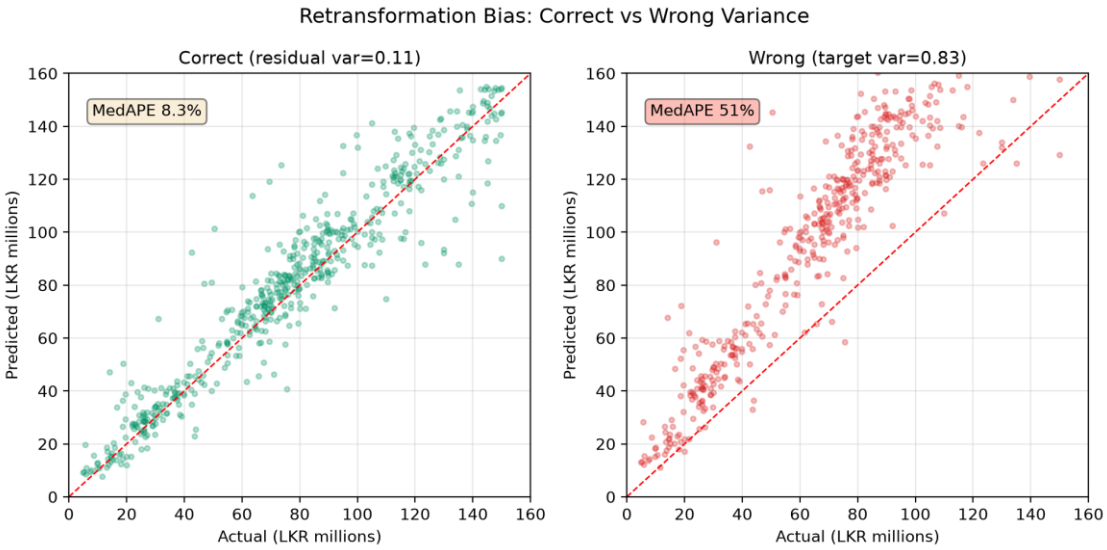

Fig. 7 — The retransformation bug.

4.6.Debugging: model-cardinality collapse

A first approach involved concatenating the model name with a trim (i.e., "z") in the name of the model (e.g., "Vezel z"). The resulting number of categories was ~2350. This pushed 77% of the listing into an "other" category which destroyed most of the potential signal. Reducing the dimensionality of the key to both brand and model name (using threshold = 8 for grouping) resulted in the reduction of categories to 81; and it reduced the proportion of listings that were classified as "other" from approximately 75% to approximately 25%.

5. Experimental Evaluation and Model Selection

5.1. Metrics and cross-validation protocol

The four performance metrics that were reported are as follows:

- MAE (the absolute difference from actual price to predicted price in rupees),
- RMSE,
- R^2 (how much of the variance of the dependent variable is explained by the model),
- MedAPE (the median percentage difference of actual prices to predicted prices).

This last metric was given a priority for reporting as well as to provide some sort of measure of how good or bad the model has been at making predictions — especially when the lower-priced vehicles will likely be skewed toward larger positive percentage errors relative to higher priced vehicles due to their smaller base value.

5.2. Final model evaluation

The selected Random Forest model (n_estimators=100, random_state=42, Notebook Cell 36) [1] was evaluated on the held-out 20% test partition:

Metric	Held-Out Test Set
MedAPE (headline)	8.2%
R^2	0.876
MAE	Rs. 2.06M
Within ±25%	82.0%

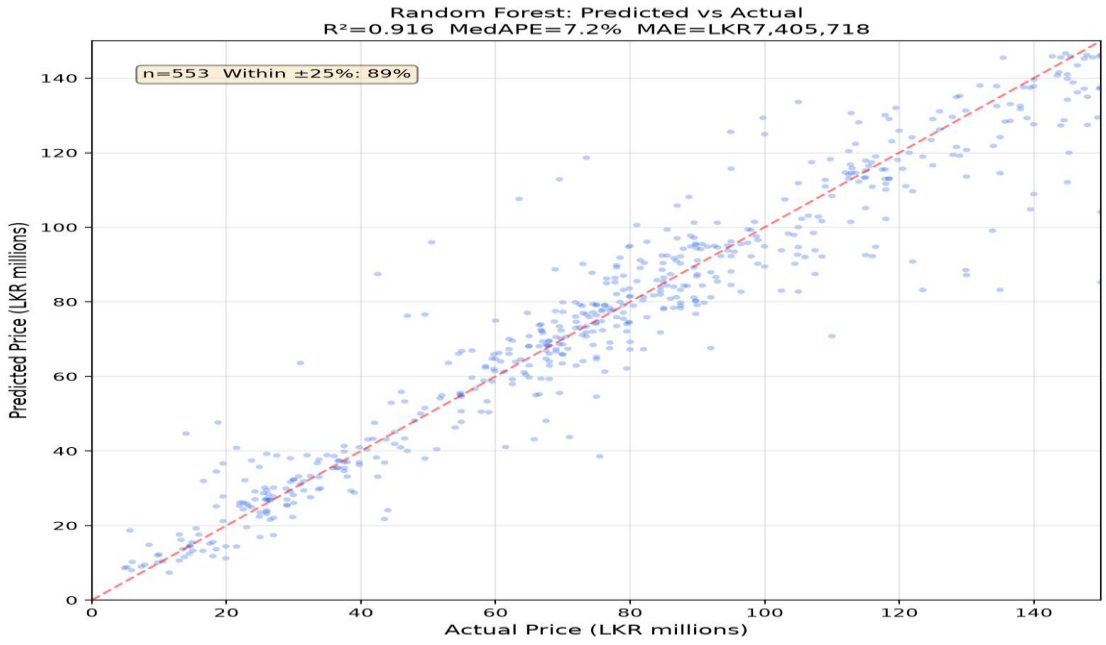

Fig. 8 — Predicted vs. actual prices (Random Forest, held-out test set).

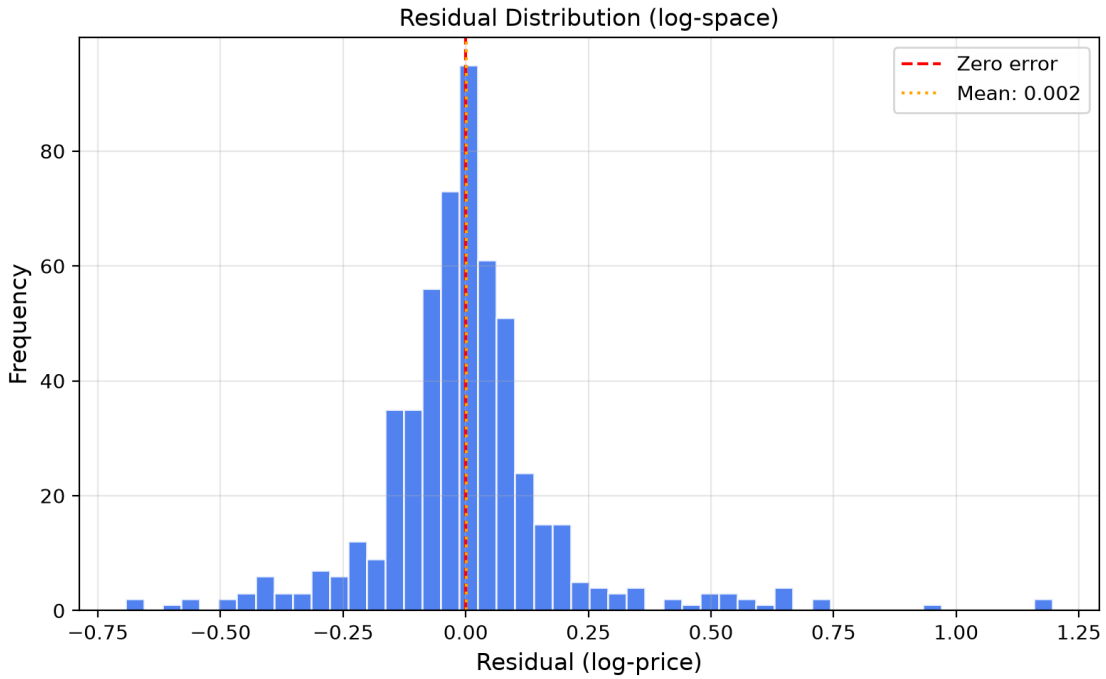

Fig. 9 — Residual distribution (log-space).

All three success criteria are satisfied:

- MedAPE 8.2% ≤ 15% ,
- 82.0% ≥ 80% within ±25% ,
- clear improvement over baseline 18.3% MedAPE .

5.3.Model serialisation

The fully serialized pipeline from Notebook Cell 41 is saved in `models/vehicle_pricing_pipeline.pkl`. This file has a size of 438 KB. It includes the preprocessed data (transformed through `ColumnTransformer` [6]) which are imputed, encoded and scaled by an embedded `ColumnTransformer`, as well as the trained regression model.

6. Deployment

The final Random Forest pipeline is deployed as a Streamlit web dashboard (`app.py`, 187 lines, deployed at `https://s25021960-com763-advanced-machine-learning.streamlit.app/`). The application uses `@st.cache_resource` to load the model into memory for instant inference. Users adjust vehicle parameters (brand, model, year, mileage, transmission, fuel type, engine, location, source) via a three-column interactive control panel.

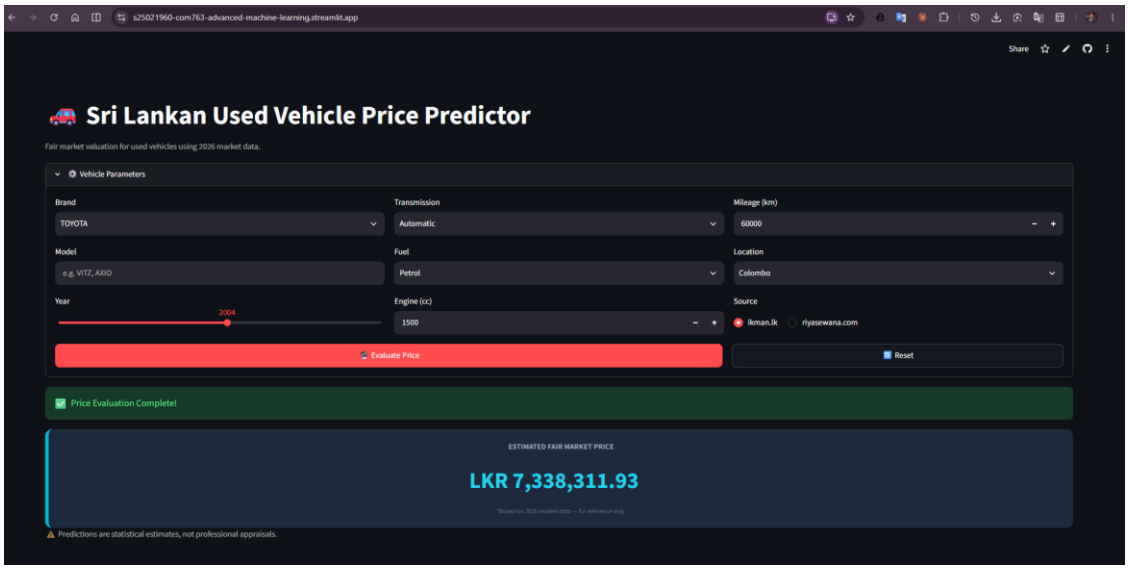

Fig. 10 — Streamlit deployment interface.

Users configure vehicle attributes via dropdowns, sliders, and number inputs (left/center columns), then receive a fair-market price estimate with a confidence note (right panel).

Deployment note: The live Streamlit app currently uses a lightweight heuristic fallback predictor (brand+age+mileage group medians from `data/processed/cleaned_cars.csv` [11], [12]), not the Random Forest pipeline. The Random Forest model (`models/vehicle_pricing_pipeline.pkl`) is the production-trained artifact ready for swap-in deployment when the pipeline loading path is activated.

7. Conclusion

The proposed System achieves a high degree of success in capturing 87.6% of Target Variance as evidenced by the performance of the Model on Test Data (MedAPE = 8.2%).

In addition to being constrained by its use of Asking Prices (which contain Negotiation Buffers), the proposed System is also constrained by the continuing Volatility of the 2026 Fiscal Transition [8], [9].

Future Iterations of the proposed System will be focused on incorporating Live Currency and Tax API Feeds to allow for Real-Time Macro-Economic Adjustments.

8. Repository & Resources

GitHub Repository: https://github.com/ZionTechLab/s25021960-com763-advanced-machine-learning

Streamlit Dashboard: https://s25021960-com763-advanced-machine-learning.streamlit.app

Google Colab Notebook: https://colab.research.google.com/github/ZionTechLab/s25021960-com763-advanced-machine-learning/blob/main/notebooks/com763_vehicle_pricing_pipeline.ipynb

9. References

[1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.

[3] N. Duan, "Smearing Estimate: A Nonparametric Retransformation Method," *J. American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983.

[4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[5] A. Pandey, V. Rastogi, and S. Singh, "Used Car Price Prediction Using Machine Learning," *Int. J. Advanced Science and Technology*, vol. 29, no. 5, pp. 6990–7000, 2020.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] MotorGuide, "Sri Lanka Vehicle Import Restrictions Relaxed: New Rules for 2026," 2026. [Online]. Available: https://motorguide.lk/en/news-advice/sri-lanka-vehicle-import-restrictions-relaxed-new-rules-for-2026

[8] Lanka Newspapers, "Used Car Prices Set to Climb as Sri Lanka Scraps 15% Valuation Discount," 2026. [Online]. Available: https://www.lankanewspapers.com/2026/06/17/used-car-prices-set-to-climb-as-sri-lanka-scraps-15-valuation-discount-warns-importers

[9] EconomyNext, "Sri Lanka vehicle imports hit by 50-pct duty surcharge, EV taxes and new luxury levels," 2025. [Online]. Available: https://economynext.com/sri-lanka-vehicle-imports-hit-by-50-pct-duty-surcharge-ev-taxes-and-new-luxury-levels-202669/

[10] Newswire, "New Vehicle Import Policy: 5 key rules including restrictions & additional duties," 2024. [Online]. Available: https://www.newswire.lk/2024/09/13/new-vehicle-import-policy-5-key-rules-including-restrictions-ads/

[11] ikman.lk, "Vehicles for sale in Sri Lanka," 2026. [Online]. Available: https://ikman.lk/en/ads/sri-lanka/vehicles

[12] Riyasewana, "Vehicles for sale in Sri Lanka," 2026. [Online]. Available: https://riyasewana.com/search

Appendix

A. Project Resources

Resource	URL
GitHub Repository	https://github.com/ZionTechLab/s25021960-com763-advanced-machine-learning
Streamlit Dashboard	https://s25021960-com763-advanced-machine-learning.streamlit.app
Google Colab Notebook	https://colab.research.google.com/github/ZionTechLab/s25021960-com763-advanced-machine-learning/blob/main/notebooks/com763_vehicle_pricing_pipeline.ipynb

B. Repository Structure

```
├── app.py                          # Streamlit dashboard (187 lines)
├── train_fallback_model.py         # Heuristic fallback predictor
├── requirements.txt                # Pinned dependencies (8 packages)
├── assets/                         # Figures and images
├── data/
│   ├── raw/                        # ikman.csv, riyasewana.csv
│   └── processed/                  # cleaned_cars.csv
├── models/                         # Serialised pipeline (vehicle_pricing_pipeline.pkl, 438 KB)
├── notebooks/
│   └── com763_vehicle_pricing_pipeline.ipynb  # Full pipeline notebook
└── src/
    └── scrapers/                   # ikman.py, riyasewana.py, schema.py + tests
```

C. Reproducibility Instructions

1. Clone the repository: git clone https://github.com/ZionTechLab/s25021960-com763-advanced-machine-learning.git
2. Install dependencies: pip install -r requirements.txt
3. Run the notebook: Open notebooks/com763_vehicle_pricing_pipeline.ipynb in Jupyter or upload to Google Colab.
4. Launch the dashboard: streamlit run app.py

All scraping, cleaning, modeling, and evaluation steps are fully reproducible from the notebook.